

附录

A 支撑材料目录

支撑材料按照完整可运行源程序与数据、正文未展示的补充图表、复现与校验材料三类整理。正文已经展示的图表不再作为独立附件重复提交，源程序运行必需的数据仍保留在完整运行目录中。目录结构如图 A1 所示。

支撑材料目录	
支撑材料 > 匿名化提交目录	
☐	00_支撑材料说明.md
☒	01_完整可运行源程序与数据
☒	08_results
☒	q1 问题一源程序、数据与结果
☒	q2 问题二源程序、数据与结果
☒	q3 问题三源程序、数据与结果
☒	q4 问题四源程序、数据与结果
☒	q5 问题五源程序、数据与结果
☐	requirements.txt
☒	02_正文未展示的补充图表
☐	附件01-07_正文未展示的补充图.pdf
☐	附件08-16_补充验证与中间结果.csv
☒	03_复现与校验材料
☐	附件17_问题二至问题五_复现检查清单.csv
☐	附件18_问题二至问题五_复现证据.json
☐	附件19_问题二至问题五_复现报告.md
☐	附件20_复现材料说明.md
☐	支撑材料目录.csv
☐	文件清单_SHA256.csv

图 A1: 支撑材料匿名化目录结构

B 核心代码

本节代码直接引自结果包的登记入口，而非重新编写的示意程序。为兼顾复核完整性与篇幅，每问收录一份承担主计算流程的脚本。代码中的固定随机种子、合法域检查和输出文件名均予以保留。

B.1 问题一：主效应、交互效应与贡献率

该程序先验证有针肋样本是否构成无重复全因子设计，再分别输出边际主效应、二阶交互残差和确定性平方和贡献率。由于没有独立重复，程序显式标记禁止进行经典显著性检验。

Listing 1: Q1 效应分析完整入口

```
1  """Q1 效应分析：计算主效应、交互效应和描述性平方和分解。"""
2  from __future__ import annotations
3
4  from itertools import combinations
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9
10
11  PACKAGE = Path(__file__).resolve().parents[1]
12  INPUT = PACKAGE / "data" / "q1_response_data.csv"
13  OUTPUT = PACKAGE / "data"
14  FACTORS = ("a", "b", "N")
15  RESPONSES = ("R", "P", "U")
16
17
18  def read_pin_data() -> pd.DataFrame:
19      data = pd.read_csv(INPUT)
20      required = {"a", "b", "N", *RESPONSES}
21      missing = required.difference(data.columns)
22      if missing:
23          raise ValueError(f"Q1 input is missing columns: {sorted(missing)}")
24      pin = data.loc[data["N"] > 0, [*FACTORS, *RESPONSES]].copy()
25      expected = int(np.prod([pin[f].nunique() for f in FACTORS]))
26      if len(pin) != expected or pin.duplicated(list(FACTORS)).any():
27          raise ValueError("The pin-fin data must be a complete, unreplicated factorial design")
28      return pin
29
30
31  def main_effects(pin: pd.DataFrame) -> pd.DataFrame:
32      rows: list[dict[str, float | str]] = []
33      for response in RESPONSES:
34          overall = float(pin[response].mean())
```

```

35     for factor in FACTORS:
36         for level, values in pin.groupby(factor, sort=True)[response]:
37             marginal = float(values.mean())
38             rows.append(
39                 {
40                     "response": response,
41                     "factor": factor,
42                     "level": float(level),
43                     "marginal_mean": marginal,
44                     "overall_mean": overall,
45                     "relative_deviation_pct": 100.0 * (marginal - overall) /
46                         overall,
47                     "sample_count": int(values.size),
48                 }
49             )
50     return pd.DataFrame(rows)
51
52 def interaction_effects(pin: pd.DataFrame) -> pd.DataFrame:
53     rows: list[dict[str, float | str]] = []
54     for response in RESPONSES:
55         overall = float(pin[response].mean())
56         one_way = {factor: pin.groupby(factor)[response].mean() for factor in
57             FACTORS}
58         for first, second in combinations(FACTORS, 2):
59             grouped = pin.groupby([first, second], sort=True)[response]
60             for (level_first, level_second), values in grouped:
61                 cell_mean = float(values.mean())
62                 interaction = (
63                     cell_mean
64                     - float(one_way[first].loc[level_first])
65                     - float(one_way[second].loc[level_second])
66                     + overall
67                 )
68                 rows.append(
69                     {
70                         "response": response,
71                         "factor_first": first,
72                         "factor_second": second,
73                         "level_first": float(level_first),
74                         "level_second": float(level_second),

```

```

74         "cell_mean": cell_mean,
75         "interaction_effect": interaction,
76         "sample_count": int(values.size),
77     }
78 )
79 return pd.DataFrame(rows)
80
81
82 def effect_contributions(pin: pd.DataFrame) -> pd.DataFrame:
83     levels = {factor: sorted(pin[factor].unique()) for factor in FACTORS}
84     counts = {factor: len(values) for factor, values in levels.items()}
85     rows: list[dict[str, float | int | str]] = []
86     for response in RESPONSES:
87         grand = float(pin[response].mean())
88         total_ss = float(np.square(pin[response] - grand).sum())
89         means = {factor: pin.groupby(factor)[response].mean() for factor in
90                 FACTORS}
91         fitted = np.full(len(pin), grand, dtype=float)
92
93         for factor in FACTORS:
94             multiplier = len(pin) // counts[factor]
95             ss = multiplier * sum((float(means[factor].loc[level]) - grand) ** 2
96                                 for level in levels[factor])
97             df = counts[factor] - 1
98             fitted += pin[factor].map(means[factor]).to_numpy(float) - grand
99             rows.append(_contribution_row(response, factor, ss, total_ss, df))
100
101         for first, second in combinations(FACTORS, 2):
102             pair_mean = pin.groupby([first, second])[response].mean()
103             multiplier = len(pin) // (counts[first] * counts[second])
104             ss = 0.0
105             contribution = np.empty(len(pin), dtype=float)
106             for pos, record in enumerate(pin[[first, second]].itertuples(index=
107                 False, name=None)):
108                 level_first, level_second = record
109                 effect = (
110                     float(pair_mean.loc[level_first, level_second])
111                     - float(means[first].loc[level_first])
112                     - float(means[second].loc[level_second])
113                     + grand
114                 )

```

```

112         contribution[pos] = effect
113     for level_first in levels[first]:
114         for level_second in levels[second]:
115             effect = (
116                 float(pair_mean.loc[level_first, level_second])
117                 - float(means[first].loc[level_first])
118                 - float(means[second].loc[level_second])
119                 + grand
120             )
121             ss += multiplier * effect**2
122             fitted += contribution
123             df = (counts[first] - 1) * (counts[second] - 1)
124             rows.append(_contribution_row(response, f"{first}x{second}", ss,
125                                         total_ss, df))
126
127 remainder = float(np.square(pin[response].to_numpy(float) - fitted).sum
128 ())
129 remainder_df = int(np.prod([counts[factor] - 1 for factor in FACTORS]))
130 rows.append(_contribution_row(response, "axb xN remainder".replace(" ",
131                             ""), remainder, total_ss, remainder_df))
132
133 result = pd.DataFrame(rows)
134 result["analysis_scope"] = "descriptive_unreplicated_factorial"
135 result["residual_df"] = 0
136 result["significance_test_allowed"] = False
137 return result
138
139 def _contribution_row(response: str, effect: str, ss: float, total_ss: float,
140 df: int) -> dict[str, float | int | str]:
141     return {
142         "response": response,
143         "effect": effect,
144         "sum_of_squares": float(ss),
145         "contribution_ratio": float(ss / total_ss if total_ss else np.nan),
146         "degrees_of_freedom": int(df),
147     }
148
149 def main() -> None:
150     pin = read_pin_data()

```

```

149     OUTPUT.mkdir(parents=True, exist_ok=True)
150     outputs = {
151         OUTPUT / "q1_main_effects.csv": main_effects(pin),
152         OUTPUT / "q1_interaction_effects.csv": interaction_effects(pin),
153         OUTPUT / "q1_effect_contributions.csv": effect_contributions(pin),
154     }
155     for path, frame in outputs.items():
156         frame.to_csv(path, index=False, encoding="utf-8-sig")
157         print(f"wrote {len(frame)} rows to {path}")
158
159
160 if __name__ == "__main__":
161     main()

```

B.2 问题二：分支代理与嵌套交叉验证

该程序对有针肋分支执行外层 5 折、重复 10 次的嵌套交叉验证，并在训练折内部选择 Ridge 正则参数；无针肋分支单独比较离散节点、线性插值与 PCHIP，避免跨结构分支插值。

Listing 2: Q2 分支代理构建与验证完整入口

```

1  """Q2 代理模型分析：嵌套交叉验证、阶数选择和无针肋插值比较。"""
2  from __future__ import annotations
3
4  from collections import Counter
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9  from scipy.interpolate import PchipInterpolator
10 from sklearn.linear_model import Ridge
11 from sklearn.model_selection import GridSearchCV, KFold, RepeatedKFold
12 from sklearn.pipeline import Pipeline
13 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
14
15
16 RESULTS = Path(__file__).resolve().parents[2]
17 INPUT = RESULTS / "q1" / "data" / "q1_response_data.csv"
18 PARETO = RESULTS / "q3" / "data" / "q3_pareto_global.csv"
19 OUTPUT = RESULTS / "q2" / "data"
20 RESPONSES = ("R", "P", "U")

```

```

21 DEGREES = (2, 3, 4)
22 ALPHAS = np.logspace(-8, 2, 11)
23 OUTER_SPLITS = 5
24 OUTER_REPEATS = 10
25 SEED = 20260730
26
27
28 def read_data() -> tuple[pd.DataFrame, pd.DataFrame]:
29     data = pd.read_csv(INPUT)
30     return data[data.N > 0].copy(), data[data.N == 0].sort_values("b").copy()
31
32
33 def pipeline(degree: int) -> Pipeline:
34     return Pipeline(
35         [
36             ("input_scale", StandardScaler()),
37             ("poly", PolynomialFeatures(degree, include_bias=False)),
38             ("feature_scale", StandardScaler()),
39             ("ridge", Ridge()),
40         ]
41     )
42
43
44 def nested_cv(pin: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.
    DataFrame]:
45     x = pin[["a", "b", "N"]].to_numpy(float)
46     outer = RepeatedKFold(n_splits=OUTER_SPLITS, n_repeats=OUTER_REPEATS,
        random_state=SEED)
47     splits = list(outer.split(x))
48     fold_rows: list[dict[str, float | int | str]] = []
49     for response in RESPONSES:
50         y = pin[response].to_numpy(float)
51         response_range = float(np.ptp(y))
52         for degree in DEGREES:
53             for fold, (train, test) in enumerate(splits):
54                 inner = KFold(n_splits=5, shuffle=True, random_state=SEED + fold)
55                 search = GridSearchCV(
56                     pipeline(degree),
57                     {"ridge__alpha": ALPHAS},
58                     scoring="neg_mean_squared_error",
59                     cv=inner,

```

```

60         n_jobs=1,
61         refit=True,
62     )
63     search.fit(x[train], y[train])
64     prediction = search.predict(x[test])
65     error = y[test] - prediction
66     fold_rows.append(
67         {
68             "response": response,
69             "degree": degree,
70             "outer_fold": fold,
71             "outer_repeat": fold // OUTER_SPLITS,
72             "test_size": len(test),
73             "selected_alpha": float(search.best_params_["ridge__alpha"
74                                     ]),
75             "rmse": float(np.sqrt(np.mean(np.square(error)))),
76             "nrmse": float(np.sqrt(np.mean(np.square(error))) /
77                             response_range),
78             "max_absolute_error": float(np.max(np.abs(error))),
79         }
80     )
81 folds = pd.DataFrame(fold_rows)
82 summary = (
83     folds.groupby(["response", "degree"], as_index=False)
84     .agg(
85         nrmse_mean=("nrmse", "mean"),
86         nrmse_std=("nrmse", "std"),
87         nrmse_max=("nrmse", "max"),
88         max_absolute_error=("max_absolute_error", "max"),
89         alpha_median=("selected_alpha", "median"),
90         alpha_min=("selected_alpha", "min"),
91         alpha_max=("selected_alpha", "max"),
92     )
93     .sort_values(["response", "nrmse_mean"])
94 )
95 alpha_rows: list[dict[str, float | int | str]] = []
96 for (response, degree), group in folds.groupby(["response", "degree"]):
97     counts = Counter(group.selected_alpha)
98     for alpha, count in sorted(counts.items()):
99         alpha_rows.append(

```



```

99         "response": response,
100         "degree": int(degree),
101         "alpha": float(alpha),
102         "selection_count": int(count),
103         "selection_share": float(count / len(group)),
104     }
105 )
106 return folds, summary, pd.DataFrame(alpha_rows)
107
108
109 def baseline_interpolation_comparison(base: pd.DataFrame) -> pd.DataFrame:
110     pareto = pd.read_csv(PARETO)[["R", "P", "U"]].to_numpy(float)
111     b_dense = np.linspace(float(base.b.min()), float(base.b.max()), 2001)
112     rows: list[dict[str, float | int | str | bool]] = []
113     methods = {
114         "discrete_nodes": (base.b.to_numpy(float), base[list(RESPONSES)].
115                             to_numpy(float)),
116         "linear": (
117             b_dense,
118             np.column_stack([np.interp(b_dense, base.b, base[response]) for
119                               response in RESPONSES]),
120         ),
121         "pchip": (
122             b_dense,
123             np.column_stack([PchipInterpolator(base.b, base[response])(b_dense)
124                               for response in RESPONSES]),
125         ),
126     }
127     for method, (b_values, values) in methods.items():
128         dominated_count = 0
129         minimum_margin = np.inf
130         for b_value, candidate in zip(b_values, values):
131             differences = candidate[None, :] - pareto
132             dominated = np.all(differences >= -1e-12, axis=1) & np.any(
133                 differences > 1e-12, axis=1)
134             if dominated.any():
135                 dominated_count += 1
136                 minimum_margin = min(minimum_margin, float(np.max(np.min(
137                     differences[dominated], axis=1))))
138         rows.append(
139             {

```

```

135         "method": method,
136         "b": float(b_value),
137         "R": float(candidate[0]),
138         "P": float(candidate[1]),
139         "U": float(candidate[2]),
140         "dominated_by_pin_pareto": bool(dominated.any()),
141     }
142 )
143 rows.append(
144     {
145         "method": f"{method}_summary",
146         "b": np.nan,
147         "R": np.nan,
148         "P": np.nan,
149         "U": np.nan,
150         "dominated_by_pin_pareto": bool(dominated_count == len(values)),
151         "points": int(len(values)),
152         "dominated_points": int(dominated_count),
153         "minimum_dominance_margin": float(minimum_margin if np.isfinite(
154             minimum_margin) else np.nan),
155     }
156 )
157 return pd.DataFrame(rows)
158
159 def main() -> None:
160     pin, base = read_data()
161     OUTPUT.mkdir(parents=True, exist_ok=True)
162     folds, summary, alpha = nested_cv(pin)
163     baseline = baseline_interpolation_comparison(base)
164     outputs = {
165         OUTPUT / "q2_nested_cv_folds.csv": folds,
166         OUTPUT / "q2_nested_model_comparison.csv": summary,
167         OUTPUT / "q2_alpha_stability.csv": alpha,
168         OUTPUT / "q2_baseline_interpolation_comparison.csv": baseline,
169     }
170     for path, frame in outputs.items():
171         frame.to_csv(path, index=False, encoding="utf-8-sig")
172         print(f"wrote {len(frame)} rows to {path}")
173
174

```

```

175 if __name__ == "__main__":
176     main()

```

B.3 问题三：Pareto 前沿与名义折中

该程序实现三次 Ridge/PCHIP 分段预测、三目标非支配筛选、分层致密网格、锚点归一化、超体积计算和增强切比雪夫折中选择。NSGA-II 重复搜索与连续精化分别保存在同目录的算法比较和精化脚本中。

Listing 3: Q3 Pareto 优化完整入口

```

1  """APMCM B题问题3：分段代理模型下的多目标优化。
2
3  程序从附件2重新拟合问题2确定的三次 Ridge 模型，并把无针肋数据
4  作为独立 PCHIP 支路处理。
5  """
6  from __future__ import annotations
7
8  import json
9  import time
10 from pathlib import Path
11
12 import matplotlib.pyplot as plt
13 import numpy as np
14 import pandas as pd
15 from matplotlib.lines import Line2D
16 from scipy.interpolate import PchipInterpolator
17 from sklearn.linear_model import Ridge
18 from sklearn.pipeline import make_pipeline
19 from sklearn.preprocessing import PolynomialFeatures, StandardScaler
20
21 ROOT = Path(__file__).resolve().parents[3]
22 PACKAGE = Path(__file__).resolve().parents[1]
23 DATA = PACKAGE / "data"
24 FIG = PACKAGE / "figures"
25 LOG = ROOT / "00_progress" / ".working" / "logs" / "q3"
26 SOURCE = ROOT / "03_data" / "raw" / "apmcm-2026-b-appendix-2-data.xlsx"
27 N_LEVELS = np.array([2, 4, 6, 8, 10], dtype=float)
28 RESP = ["R", "P", "U"]
29
30
31 def read_data() -> tuple[pd.DataFrame, pd.DataFrame]:

```

```

32 raw = pd.read_excel(SOURCE, header=1)
33 raw.columns = ["id", "a", "b", "N", "R", "P", "U"]
34 raw = raw.apply(pd.to_numeric, errors="coerce").dropna().reset_index(drop=
    True)
35 return raw[raw.N > 0].copy(), raw[raw.N == 0].copy()
36
37
38 def build_surrogate(pin: pd.DataFrame, base: pd.DataFrame):
39     x = pin[["a", "b", "N"]].to_numpy(float)
40     # 问题2的最终设定: 输入标准化 -> 完整三次基 -> 特征标准化 -> Ridge。
41     ridge = {}
42     for y, alpha in zip(Resp, [1e-5, 1e-2, 1e-3]):
43         ridge[y] = make_pipeline(StandardScaler(), PolynomialFeatures(3,
            include_bias=False),
44                                 StandardScaler(), Ridge(alpha=alpha)).fit(x, pin[y
                ])
45     pchip = {y: PchipInterpolator(base.b.to_numpy(float), base[y].to_numpy(
        float)) for y in Resp}
46     return ridge, pchip
47
48
49 def predict(x: np.ndarray, ridge, pchip) -> np.ndarray:
50     """批量分段预测; x 的列依次为 a,b,N。"""
51     x = np.asarray(x, dtype=float)
52     ans = np.empty((len(x), 3))
53     base = (np.isclose(x[:, 0], 0) & np.isclose(x[:, 2], 0))
54     if base.any():
55         ans[base] = np.column_stack([pchip[y](x[base, 1]) for y in Resp])
56     if (~base).any():
57         ans[~base] = np.column_stack([ridge[y].predict(x[~base]) for y in Resp])
58     return ans
59
60
61 def nondominated(f: np.ndarray, tol: float = 1e-11) -> np.ndarray:
62     """三目标最小化的  $O(n \log n)$  扫描, 返回非支配点布尔掩码。"""
63     if f.shape[1] == 2:
64         order = np.lexsort((f[:, 1], f[:, 0])); keep = np.zeros(len(f), dtype=
            bool); best = np.inf
65         for idx in order:
66             if f[idx, 1] < best - tol:
67                 keep[idx] = True; best = f[idx, 1]

```

```

68     return keep
69     order = np.lexsort((f[:, 2], f[:, 1], f[:, 0]))
70     pvals = np.unique(f[:, 1]); m = len(pvals)
71     tree = np.full(m + 2, np.inf)
72     keep = np.zeros(len(f), dtype=bool)
73     for idx in order:
74         pos = np.searchsorted(pvals, f[idx, 1]) + 1
75         q, best = pos, np.inf
76         while q:
77             best = min(best, tree[q]); q -= q & -q
78             if best > f[idx, 2] + tol:
79                 keep[idx] = True
80             q = pos
81             while q <= m:
82                 tree[q] = min(tree[q], f[idx, 2]); q += q & -q
83     return keep
84
85
86 def grid_search(ridge, pchip, n_grid: int = 401) -> tuple[np.ndarray, np.
    ndarray, dict]:
87     a = np.linspace(.10, .30, n_grid); b = np.linspace(3., 4.5, n_grid)
88     aa, bb = np.meshgrid(a, b, indexing="ij")
89     designs, values = [], []
90     layers = {}
91     for n in N_LEVELS:
92         x = np.column_stack([aa.ravel(), bb.ravel(), np.full(aa.size, n)])
93         f = predict(x, ridge, pchip); take = nondominated(f)
94         designs.append(x[take]); values.append(f[take])
95         layers[str(int(n))] = int(take.sum())
96     x0 = np.column_stack([np.zeros(2001), np.linspace(3., 4.5, 2001), np.zeros
        (2001)])
97     f0 = predict(x0, ridge, pchip); take0 = nondominated(f0)
98     designs.append(x0[take0]); values.append(f0[take0])
99     x, f = np.vstack(designs), np.vstack(values)
100    take = nondominated(f)
101    return x[take], f[take], {"grid": n_grid, "layer_nondominated": layers, "
        baseline_kept": int(take0.sum())}
102
103
104 def anchors(x, f):
105     ids = np.argmin(f, axis=0)

```

```

106     ideal = f[ids, np.arange(3)]
107     nadir = f[ids].max(axis=0)
108     return ids, ideal, nadir
109
110
111 def normalize(f, ideal, nadir):
112     return (f - ideal) / np.maximum(nadir - ideal, 1e-12)
113
114
115 def hypervolume3(z: np.ndarray, ref=np.array([1.10, 1.10, 1.10])) -> float:
116     # 精确切片计算三维 HV; 候选前沿通常很小。
117     z = z[np.all(z < ref, axis=1)]
118     if len(z) == 0: return 0.0
119     z = z[nondominated(z)]
120     xs = np.unique(np.r_[z[:, 0], ref[0]]); hv = 0.0
121     for lo, hi in zip(xs[:-1], xs[1:]):
122         yz = z[z[:, 0] <= lo[:, 1:]]
123         if not len(yz): continue
124         yz = yz[nondominated(yz)]
125         yy = np.unique(np.r_[yz[:, 0], ref[1]])
126         area = 0.0
127         for yl, yh in zip(yy[:-1], yy[1:]):
128             us = yz[yz[:, 0] <= yl, 1]
129             area += (yh - yl) * max(0.0, ref[2] - np.min(us))
130         hv += (hi - lo) * area
131     return float(hv)
132
133
134 def candidate_table(x, f, pin, ideal, nadir):
135     z = normalize(f, ideal, nadir); score = z.max(1) + 1e-4 * z.sum(1); l2 = np
136         .sqrt((z*z).mean(1))
137     labels = ["最小热阻锚点", "最小压降锚点", "最小非均匀性锚点", "连续综合最优",
138         "最近理想点"]
139     ind = list(np.argmin(f, 0)) + [int(np.argmin(score)), int(np.argmin(l2))]
140     # 原始有针肋样本按同一折中准则加入。
141     sf = pin[RESP].to_numpy(float); sz = normalize(sf, ideal, nadir)
142     si = int(np.argmin(sz.max(1) + 1e-4*sz.sum(1)))
143     px = pin.iloc[si][["a", "b", "N"]].to_numpy(float)
144     rows = []
145     for lab, i in zip(labels, ind): rows.append([lab, *x[i], *f[i], score[i],
146         l2[i]])

```

```

144     rows.append(["最佳原始样本", *px, *sf[si], sz[si].max()+1e-4*sz[si].sum(),
145                 np.sqrt((sz[si]**2).mean())])
146     columns = ["方案", "a", "b", "N", "R", "P", "U", "C_AT", "C2"]
147     return pd.DataFrame(rows, columns=columns), int(np.argmin(score))
148
149 def make_figures(x, f, candidates, ideal, nadir):
150     plt.rcParams.update({"font.sans-serif":["Microsoft YaHei", "SimHei", "
151         DejaVu Sans"], "axes.unicode_minus":False, "font.size":9})
152     z = normalize(f, ideal, nadir); colors = {2:"#2563eb",4:"#0891b2",6:"#16
153         a34a",8:"#d97706",10:"#dc2626",0:"#6b7280"}
154     fig = plt.figure(figsize=(10.6, 7.4)); ax3 = fig.add_subplot(221,
155         projection="3d")
156     for n in [0,2,4,6,8,10]:
157         q = np.isclose(x[:,2], n); ax3.scatter(f[q,0],f[q,1],f[q,2],s=10,c=
158             colors[n],label=f"N={n}",alpha=.78)
159     ax3.set(xlabel="R",ylabel="P",zlabel="U"); ax3.legend(fontsize=7, ncol=2)
160     for ax,(i,j),name in zip([fig.add_subplot(222),fig.add_subplot(223),fig.
161         add_subplot(224)],[(0,1),(0,2),(1,2)],["R-P","R-U","P-U"]):
162         for n in [0,2,4,6,8,10]:
163             q=np.isclose(x[:,2],n); ax.scatter(f[q,i],f[q,j],s=10,c=colors[n],
164                 alpha=.8)
165         ax.set(xlabel=name.split('-')[0],ylabel=name.split('-')[1]); ax.grid(
166             alpha=.22)
167     fig.tight_layout(); fig.savefig(FIG/"fig_q3_01_pareto_front.pdf",
168         bbox_inches="tight"); plt.close(fig)
169     # 平行坐标: 选择表中核心候选, 使用统一标尺。
170     fig, ax = plt.subplots(figsize=(9.0,4.2)); vals=normalize(candidates[RESP].
171         to_numpy(float),ideal,nadir)
172     for i,row in enumerate(vals): ax.plot(range(3),row,marker="o",lw=1.4,label=
173         candidates.iloc[i,0])
174     ax.set_xticks(range(3),["R","P","U"]); ax.set_ylabel("统一归一化性能"); ax.
175         grid(alpha=.25); ax.legend(ncol=2,fontsize=8); fig.tight_layout(); fig.
176         savefig(FIG/"fig_q3_02_candidate_comparison.pdf",bbox_inches="tight");
177         plt.close(fig)
178     # 最终点所属 N 切片的折中评分图。
179     best=candidates[candidates["方案"]=="连续综合最优"].iloc[0]; n=best.N
180     aa=np.linspace(.1,.3,151); bb=np.linspace(3,4.5,151); A,B=np.meshgrid(aa,bb
181         ,indexing="ij")
182     xx=np.column_stack([A.ravel(),B.ravel(),np.full(A.size,n)]); zz=normalize(
183         predict(xx, RIDGE, PCHIP),ideal,nadir); s=zz.max(1)+1e-4*zz.sum(1)

```

```

169     fig,ax=plt.subplots(figsize=(6.4,4.8)); cf=ax.contourf(A,B,s.reshape(A.
        shape),levels=18,cmap="YlGnBu"); fig.colorbar(cf,ax=ax,label="$C_{AT}$")
170     q=np.isclose(x[:,2],n); ax.scatter(x[q,0],x[q,1],s=6,c="white",alpha=.65,
        label="该层Pareto点"); ax.scatter([best.a],[best.b],marker="*",s=170,c="
        #dc2626",edgecolors="white",label="综合最优")
171     ax.set(xlabel="a",ylabel="b",title=f"N={int(n)} 的综合评分切片"); ax.legend
        (); fig.tight_layout(); fig.savefig(FIG/"fig_q3_03_design_slice.pdf",
        bbox_inches="tight"); plt.close(fig)
172
173
174 def write_tex(summary, candidates, payoff):
175     best = candidates[candidates["方案"]=="连续综合最优"].iloc[0]
176     def table(df):
177         rows=[]
178         for _,r in df.iterrows():
179             rows.append(f"{r['方案']} & {r.a:.4f} & {r.b:.4f} & {int(r.N)} & {r.R
                :.6f} & {r.P:.6f} & {r.U:.6f} & {r.C_AT:.4f} \\\\"
180             return "\n".join(rows)
181     tex=f'''\\documentclass[UTF8,a4paper,11pt]{ctexart}
182     \\usepackage{geometry,booktabs,float,graphicx,amsmath,siunitx}
183     \\geometry{margin=2.15cm}\\title{APMCM B题问题3: 芯片热管理结构的多目标优化
        }\\author{\\}\\date{\\}
184     \\begin{document}\\maketitle
185     \\section{模型与求解方法} 本文继承问题2的分段代理模型: 有针肋区域以标准化输入、
        完整三次多项式及Ridge正则化构成响应面; 无针肋$(a,N)=(0,0)$支路以PCHIP插值单独
        处理。故在$0.10\\le a\\le 0.30, 3\\le b\\le 4.5, N\\in\\{2,4,6,8,10\\}$及无针
        肋支路的并集上, 同时最小化无量纲热阻$R$、压降$P$和温度非均匀性$U$。
186
187 对五个离散$N$水平分别以$401\\times 401$致密网格搜索, 所有候选经三目标非支配筛选后与
        2001点无针肋PCHIP支路合并。三个单目标锚点确定理想点, 锚点收益矩阵给出初始Nadir
        尺度; 最终参考前沿的覆盖检查未改变该尺度。综合决策采用等权增强切比雪夫评分
188     \\[C_{AT}]=\\max(z_R,z_P,z_U)+10^{-4}(z_R+z_P+z_U), \\quad z_k=(F_k-F_k^I)/(
        F_k^N-F_k^I).\\]
189     \\section{结果与综合方案} 本次确定性搜索得到全局Pareto点{summary['
        pareto_points']}个; 无针肋支路在全局合并后保留{summary['
        baseline_global_points']}个点。连续综合最优为$a={best.a:.4f}$, $b={best.b:.4
        f}$, $N={int(best.N)}$, 预测$(R,P,U)=(\\{best.R:.6f\\},\\{best.P:.6f\\},\\{best.U:.6f
        \\})$。它位于代理模型合法域内, 且为全局非支配解。
190     \\begin{table}[H]\\centering\\caption{单目标锚点与折中方案比较}\\small
191     \\begin{tabular}{lrrrrrrr}\\toprule 方案&a&b&N&R&P&U&$C_{AT}$\\\\
        midrule

```



```

192 {table(candidates)}\\bottomrule\\end{tabular}\\end{table}}
193 \\begin{figure}[H]\\centering\\includegraphics[width=.94\\linewidth]{../
    figures/fig_q3_01_pareto_front.pdf}\\caption{{全局Pareto前沿及两两投影}}\\
    end{figure}}
194 \\begin{figure}[H]\\centering\\includegraphics[width=.86\\linewidth]{../
    figures/fig_q3_02_candidate_comparison.pdf}\\caption{{候选方案的统一归一化
    比较}}\\end{figure}}
195 \\begin{figure}[H]\\centering\\includegraphics[width=.68\\linewidth]{../
    figures/fig_q3_03_design_slice.pdf}\\caption{{综合最优点所在排数层的局部评分
    }}\\end{figure}}
196 \\section{结论} 三个目标的冲突使单一极端点不可作为综合设计。等权增强切比雪夫准则
    选择的方案控制了最大标准化退让；它可作为名义工况基准，权重变化和扰动鲁棒性应在
    后续问题中另行分析。\\end{document}}''
197 DRAFT.mkdir(parents=True, exist_ok=True)
198 (DRAFT/"q3_demo.tex").write_text(tex,encoding="utf-8")
199
200
201 def main():
202     global RIDGE, PCHIP
203     for d in [DATA,FIG,LOG,DRAFT]: d.mkdir(parents=True,exist_ok=True)
204     pin, base=read_data(); RIDGE,PCHIP=build_surrogate(pin,base)
205     t=time.perf_counter(); x,f,grid_meta=grid_search(RIDGE,PCHIP); elapsed=time
        .perf_counter()-t
206     ids,ideal,nadir=anchors(x,f); nadir=np.maximum(nadir,f.max(0))
207     z=normalize(f,ideal,nadir); take=nondominated(f); x,f,z=x[take],f[take],z[
        take]
208     pd.DataFrame(np.c_[x,f],columns=["a","b","N","R","P","U"]).to_csv(DATA/"
        q3_pareto_global.csv",index=False)
209     payoff=pd.DataFrame(f[ids],index=["R锚点","P锚点","U锚点"],columns=RESP);
        payoff.to_csv(DATA/"q3_payoff_matrix.csv",encoding="utf-8-sig")
210     candidates,best_i=candidate_table(x,f,pin,ideal,nadir); candidates.to_csv(
        DATA/"q3_compromise_candidates.csv",index=False,encoding="utf-8-sig")
211     summary={"grid_search_seconds":elapsed,"grid_meta":grid_meta,"pareto_points
        ":int(len(f)),"baseline_global_points":int(np.sum(x[:,2]==0)),"ideal":
        ideal.tolist(),"nadir":nadir.tolist(),"hypervolume":hypervolume3(z),"
        best_index":best_i}
212     (LOG/"optimization_summary.json").write_text(json.dumps(summary,
        ensure_ascii=False,indent=2),encoding="utf-8")
213     # 演示性论文在 drafts 中独立维护；模型复现只写入结果数据产物，
214     # 避免运行数值程序时覆盖未审核的论文本。
215     print(json.dumps(summary,ensure_ascii=False,indent=2))

```

```

216
217 if __name__ == "__main__": main()

```

B.4 问题四：权重域与最小最大遗憾

该程序生成完整权重单纯形，并在完整、平衡和散热优先三个权重域内分别计算绝对遗憾与相对遗憾，输出最坏遗憾、平均遗憾、95%分位遗憾及最优排数的权重占比。

Listing 4: Q4 偏好敏感性完整入口

```

1  """Q4 偏好敏感性分析：权重域、绝对遗憾与相对遗憾比较。"""
2  from __future__ import annotations
3
4  from pathlib import Path
5
6  import numpy as np
7  import pandas as pd
8
9
10 RESULTS = Path(__file__).resolve().parents[2]
11 INPUT = RESULTS / "q3" / "data" / "q3_pareto_reference_merged.csv"
12 OUTPUT = RESULTS / "q4" / "data"
13 IDEAL = np.array([0.720227030892114, 0.076039812756808, 0.771786631558643])
14 NADIR = np.array([0.760176422366934, 0.158555235146874, 0.819442088245665])
15 SPAN = NADIR - IDEAL
16 RHO = 1e-4
17
18
19 def simplex(resolution: int) -> np.ndarray:
20     return np.array(
21         [(i / resolution, j / resolution, (resolution - i - j) / resolution) for
22          i in range(resolution + 1) for j in range(resolution + 1 - i)],
23         dtype=float,
24     )
25
26 def weighted_scores(z: np.ndarray, weight: np.ndarray) -> np.ndarray:
27     weighted = z * (3.0 * weight)
28     return np.max(weighted, axis=1) + RHO * np.sum(weighted, axis=1)
29
30
31 def domains(weights: np.ndarray) -> dict[str, np.ndarray]:

```

```

32     return {
33         "complete_simplex": weights,
34         "balanced_w_ge_0p1": weights[np.all(weights >= 0.1 - 1e-12, axis=1)],
35         "thermal_priority_wR_ge_0p5": weights[weights[:, 0] >= 0.5 - 1e-12],
36     }
37
38
39 def evaluate_domain(reference: pd.DataFrame, weights: np.ndarray, domain_name:
    str) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
40     values = reference[["R", "P", "U"]].to_numpy(float)
41     designs = reference[["a", "b", "N"]].to_numpy(float)
42     z = (values - IDEAL) / SPAN
43     n_candidates = len(reference)
44     max_abs = np.full(n_candidates, -np.inf)
45     max_rel = np.full(n_candidates, -np.inf)
46     sum_abs = np.zeros(n_candidates)
47     sum_rel = np.zeros(n_candidates)
48     mapping_rows = []
49     score_ranges = []
50
51     for weight_id, weight in enumerate(weights):
52         candidate_scores = weighted_scores(z, weight)
53         optimum_index = int(np.argmin(candidate_scores))
54         lower = float(candidate_scores[optimum_index])
55         upper = float(np.max(candidate_scores))
56         score_range = upper - lower
57         absolute = candidate_scores - lower
58         relative = absolute / (score_range + 1e-12)
59         max_abs = np.maximum(max_abs, absolute)
60         max_rel = np.maximum(max_rel, relative)
61         sum_abs += absolute
62         sum_rel += relative
63         score_ranges.append(score_range)
64         mapping_rows.append(
65             {
66                 "weight_domain": domain_name,
67                 "weight_id": weight_id,
68                 "w_R": weight[0],
69                 "w_P": weight[1],
70                 "w_U": weight[2],
71                 "a": designs[optimum_index, 0],

```

```

72         "b": designs[optimum_index, 1],
73         "N": designs[optimum_index, 2],
74         "optimal_score": lower,
75         "score_range": score_range,
76     }
77 )
78
79 summaries = []
80 for definition, maximum, total in (
81     ("absolute", max_abs, sum_abs),
82     ("relative", max_rel, sum_rel),
83 ):
84     mean = total / len(weights)
85     selected = int(np.lexsort((mean, maximum))[0])
86     selected_regrets = []
87     for weight in weights:
88         candidate_scores = weighted_scores(z, weight)
89         lower = float(np.min(candidate_scores))
90         absolute = float(candidate_scores[selected] - lower)
91         if definition == "absolute":
92             selected_regrets.append(absolute)
93         else:
94             selected_regrets.append(absolute / (float(np.max(candidate_scores)
95                 )) - lower + 1e-12))
96     summaries.append(
97         {
98             "weight_domain": domain_name,
99             "regret_definition": definition,
100             "weight_count": len(weights),
101             "a": designs[selected, 0],
102             "b": designs[selected, 1],
103             "N": designs[selected, 2],
104             "R": values[selected, 0],
105             "P": values[selected, 1],
106             "U": values[selected, 2],
107             "max_regret": maximum[selected],
108             "mean_regret": mean[selected],
109             "p95_regret": float(np.quantile(selected_regrets, 0.95)),
110             "minimum_weight_score_range": float(np.min(score_ranges)),
111             "near_zero_range_weights": int(np.sum(np.asarray(score_ranges) <
112                 1e-8)),

```

```

111         }
112     )
113
114     mapping = pd.DataFrame(mapping_rows)
115     shares = (
116         mapping.groupby("N", as_index=False)
117         .size()
118         .rename(columns={"size": "winning_weight_count"})
119     )
120     shares["weight_domain"] = domain_name
121     shares["winning_weight_share"] = shares.winning_weight_count / len(mapping)
122     return pd.DataFrame(summaries), mapping, shares
123
124
125 def main() -> None:
126     reference = pd.read_csv(INPUT)
127     all_weights = simplex(50)
128     summary_frames = []
129     mapping_frames = []
130     share_frames = []
131     for domain_name, weights in domains(all_weights).items():
132         summary, mapping, shares = evaluate_domain(reference, weights,
133             domain_name)
134         summary_frames.append(summary)
135         mapping_frames.append(mapping)
136         share_frames.append(shares)
137
138     summaries = pd.concat(summary_frames, ignore_index=True)
139     mappings = pd.concat(mapping_frames, ignore_index=True)
140     shares = pd.concat(share_frames, ignore_index=True)
141     OUTPUT.mkdir(parents=True, exist_ok=True)
142     outputs = {
143         OUTPUT / "q4_weight_domain_comparison.csv": summaries[summaries.
144             regret_definition == "absolute"].copy(),
145         OUTPUT / "q4_regret_definition_comparison.csv": summaries,
146         OUTPUT / "q4_weight_mapping.csv": mappings,
147         OUTPUT / "q4_weight_share.csv": shares,
148     }
149     for path, frame in outputs.items():
150         frame.to_csv(path, index=False, encoding="utf-8-sig")
151         print(f"wrote {len(frame)} rows to {path}")

```

```

150
151
152 if __name__ == "__main__":
153     main()

```

B.5 问题五：误差分布、模型歧义与 Sobol 分析

该程序统一生成多类加工误差样本，计算候选方案的 CVaR95，检查扰动支持集，并对六类参数化工况执行模型遗憾比较；最后以固定种子和 2^{15} 个基样本估计一阶及总效应 Sobol 指数。

Listing 5: Q5 鲁棒性与全局灵敏度完整入口

```

1  """Q5 鲁棒性分析：误差分布、模型歧义和全局敏感性。"""
2  from __future__ import annotations
3
4  import sys
5  from pathlib import Path
6
7  import numpy as np
8  import pandas as pd
9  from scipy import stats
10 from scipy.stats import qmc
11
12
13 RESULTS = Path(__file__).resolve().parents[2]
14 sys.path.insert(0, str(RESULTS / "q3" / "code"))
15 import q3_optimize_pareto as q3 # noqa: E402
16 import q5_optimize_robustness as q45 # noqa: E402
17
18
19 OUTPUT = RESULTS / "q5" / "data"
20 SCENARIO_OPTIMA = OUTPUT / "q5_operation_design_comparison.csv"
21 STRUCTURE_OPTIMA = OUTPUT / "q5_structure_layer_optima.csv"
22 N_LEVELS = (2.0, 4.0, 6.0, 8.0, 10.0)
23 TAU = 0.025
24 SEED = 20260730
25 SCENARIOS = {
26     "S1-A": ((0.80, 0.75, 0.25, 0.25), "A"),
27     "S1-B": ((0.80, 0.75, 0.25, 0.25), "B"),
28     "S2-A": ((0.50, 0.50, 0.50, 0.50), "A"),
29     "S2-B": ((0.50, 0.50, 0.50, 0.50), "B"),

```

```

30     "S3-A": ((0.20, 0.25, 0.80, 0.80), "A"),
31     "S3-B": ((0.20, 0.25, 0.80, 0.80), "B"),
32 }
33
34
35 def cvar_rows(values: np.ndarray, alpha: float = 0.95) -> np.ndarray:
36     threshold = np.quantile(values, alpha, axis=1)
37     return np.array([row[row >= limit].mean() for row, limit in zip(values,
38                             threshold)])
39
40 def candidate_designs() -> np.ndarray:
41     a = np.linspace(0.105, 0.295, 13)
42     b = np.linspace(3.0375, 4.4625, 13)
43     aa, bb = np.meshgrid(a, b, indexing="ij")
44     designs = [np.column_stack([aa.ravel(), bb.ravel(), np.full(aa.size, n)])
45               for n in N_LEVELS]
46     extra = []
47     for path in (SCENARIO_OPTIMA, STRUCTURE_OPTIMA):
48         frame = pd.read_csv(path)
49         extra.append(frame[["a", "b", "N"]].to_numpy(float))
50     all_designs = np.vstack([*designs, *extra])
51     all_designs[:, 0] = np.clip(all_designs[:, 0], 0.105, 0.295)
52     all_designs[:, 1] = np.clip(all_designs[:, 1], 3.0375, 4.4625)
53     return np.unique(np.round(all_designs, 12), axis=0)
54
55 def lhs_uniform(n: int, dimensions: int, seed: int) -> np.ndarray:
56     return qmc.LatinHypercube(dimensions, seed=seed).random(n)
57
58
59 def structural_distributions(n: int = 2048) -> dict[str, np.ndarray]:
60     base_u = lhs_uniform(n, 2, SEED)
61     uniform = qmc.scale(base_u, [-TAU, -TAU], [TAU, TAU])
62     triangular = np.column_stack(
63         [
64             np.where(
65                 base_u[:, index] < 0.5,
66                 -TAU + np.sqrt(base_u[:, index] * 2.0 * TAU**2),
67                 TAU - np.sqrt((1.0 - base_u[:, index]) * 2.0 * TAU**2),
68             )

```

```

69         for index in range(2)
70     ]
71 )
72 truncated = stats.truncnorm.ppf(base_u, -2.0, 2.0, loc=0.0, scale=TAU /
73     2.0)
74
75 normal_scores = stats.norm.ppf(np.clip(base_u, 1e-12, 1 - 1e-12))
76 correlated = {}
77 for label, rho in (("uniform_positive_rho_0p6", 0.6), ("
78     uniform_negative_rho_m0p6", -0.6)):
79     chol = np.linalg.cholesky(np.array([[1.0, rho], [rho, 1.0]]))
80     uniforms = stats.norm.cdf(normal_scores @ chol.T)
81     correlated[label] = qmc.scale(uniforms, [-TAU, -TAU], [TAU, TAU])
82
83 corners = np.array([[-TAU, -TAU], [-TAU, TAU], [TAU, -TAU], [TAU, TAU]])
84 return {
85     "independent_uniform": uniform,
86     "symmetric_triangular": triangular,
87     "truncated_normal_2sigma": truncated,
88     **correlated,
89     "deterministic_corners": corners,
90 }
91
92 def structural_risk(designs: np.ndarray, eps: np.ndarray, ridge, pchip,
93     batch_size: int = 24) -> np.ndarray:
94     risks = []
95     for start in range(0, len(designs), batch_size):
96         batch = designs[start : start + batch_size]
97         repeated = np.repeat(batch, len(eps), axis=0)
98         tiled = np.tile(eps, (len(batch), 1))
99         realized = repeated.copy()
100         realized[:, 0] += 0.20 * tiled[:, 0]
101         realized[:, 1] += 1.50 * tiled[:, 1]
102         values = q3.predict(realized, ridge, pchip)
103         scores = q45.score(values).reshape(len(batch), len(eps))
104         risks.extend(cvar_rows(scores))
105     return np.asarray(risks)
106
107 def distribution_comparison(designs: np.ndarray, ridge, pchip) -> pd.DataFrame:

```



```

107     rows = []
108     for distribution, eps in structural_distributions().items():
109         risks = structural_risk(designs, eps, ridge, pchip)
110         index = int(np.argmin(risks))
111         rows.append(
112             {
113                 "distribution": distribution,
114                 "sample_count": len(eps),
115                 "a": designs[index, 0],
116                 "b": designs[index, 1],
117                 "N": designs[index, 2],
118                 "cvar95": risks[index],
119                 "candidate_count": len(designs),
120                 "support_tau": TAU,
121             }
122         )
123     return pd.DataFrame(rows)
124
125
126 def domain_check() -> pd.DataFrame:
127     layer = pd.read_csv(STRUCTURE_OPTIMA)
128     robust = layer.iloc[int(layer.CVaR95.argmin())]
129     designs = {
130         "q3_nominal_boundary": np.array([0.2249317396262705, 4.5, 6.0]),
131         "q3_nominal_projected": np.array([0.2249317396262705, 4.4625, 6.0]),
132         "structure_robust": robust[["a", "b", "N"]].to_numpy(float),
133     }
134     rows = []
135     for name, design in designs.items():
136         minimum_a, maximum_a = design[0] - 0.005, design[0] + 0.005
137         minimum_b, maximum_b = design[1] - 0.0375, design[1] + 0.0375
138         rows.append(
139             {
140                 "candidate": name,
141                 "a": design[0],
142                 "b": design[1],
143                 "N": design[2],
144                 "min_realized_a": minimum_a,
145                 "max_realized_a": maximum_a,
146                 "min_realized_b": minimum_b,
147                 "max_realized_b": maximum_b,

```

```

148         "formal_domain_contained": bool(minimum_a >= 0.10 and maximum_a
149             <= 0.30 and minimum_b >= 3.0 and maximum_b <= 4.5),
150     }
151 )
152 return pd.DataFrame(rows)
153
154 def common_operation_samples(n: int = 1024) -> tuple[np.ndarray, np.ndarray, np
155     .ndarray]:
156     sample = lhs_uniform(n, 4, SEED + 1)
157     eps = qmc.scale(sample[:, :2], [-TAU, -TAU], [TAU, TAU])
158     sm = 0.95 + 0.10 * sample[:, 2]
159     sq = 0.95 + 0.10 * sample[:, 3]
160     return eps, sm, sq
161
162 def operation_risk(designs: np.ndarray, eps: np.ndarray, sm: np.ndarray, sq: np
163     .ndarray, ridge, pchip, pars, u_model: str, batch_size: int = 20) -> np.
164     ndarray:
165     risks = []
166     alpha, theta, exponent_r, exponent_u = pars
167     for start in range(0, len(designs), batch_size):
168         batch = designs[start : start + batch_size]
169         repeated = np.repeat(batch, len(eps), axis=0)
170         tiled_eps = np.tile(eps, (len(batch), 1))
171         tiled_sm = np.tile(sm, len(batch))
172         tiled_sq = np.tile(sq, len(batch))
173         realized = repeated.copy()
174         realized[:, 0] += 0.20 * tiled_eps[:, 0]
175         realized[:, 1] += 1.50 * tiled_eps[:, 1]
176         base = q3.predict(realized, ridge, pchip)
177         thermal = tiled_sq * base[:, 0] * (theta + (1.0 - theta) * tiled_sm **
178             (-exponent_r))
179         pressure = base[:, 1] * (alpha * tiled_sm + (1.0 - alpha) * tiled_sm**2)
180         uniformity = base[:, 2] * tiled_sm ** (-exponent_u)
181         if u_model == "B":
182             uniformity *= tiled_sq
183         scores = q45.score(np.column_stack([thermal, pressure, uniformity])).
184             reshape(len(batch), len(eps))
185         risks.extend(cvar_rows(scores))
186     return np.asarray(risks)

```

```

183
184
185 def model_ambiguity_analysis(designs: np.ndarray, ridge, pchip) -> tuple[pd.
    DataFrame, pd.DataFrame]:
186     eps, sm, sq = common_operation_samples()
187     risk_columns = {}
188     scenario_rows = []
189     for scenario, (pars, u_model) in SCENARIOS.items():
190         risks = operation_risk(designs, eps, sm, sq, ridge, pchip, pars, u_model
            )
191         optimum = float(np.min(risks))
192         risk_columns[scenario] = risks
193         index = int(np.argmin(risks))
194         scenario_rows.append(
195             {
196                 "selection": f"scenario_optimum_{scenario}",
197                 "scenario": scenario,
198                 "a": designs[index, 0],
199                 "b": designs[index, 1],
200                 "N": designs[index, 2],
201                 "cvar95": risks[index],
202             }
203         )
204
205     risk_frame = pd.DataFrame(risk_columns)
206     regrets = risk_frame - risk_frame.min(axis=0)
207     risk_frame.insert(0, "N", designs[:, 2])
208     risk_frame.insert(0, "b", designs[:, 1])
209     risk_frame.insert(0, "a", designs[:, 0])
210     risk_frame["max_model_regret"] = regrets.max(axis=1)
211     risk_frame["mean_model_regret"] = regrets.mean(axis=1)
212     robust_index = int(np.lexsort((risk_frame.mean_model_regret, risk_frame.
        max_model_regret))[0])
213     scenario_rows.append(
214         {
215             "selection": "minimax_model_regret",
216             "scenario": "all_six",
217             "a": designs[robust_index, 0],
218             "b": designs[robust_index, 1],
219             "N": designs[robust_index, 2],
220             "cvar95": np.nan,

```

```

221         "max_model_regret": risk_frame.loc[robust_index, "max_model_regret"
222         ],
223         "mean_model_regret": risk_frame.loc[robust_index, "mean_model_regret"
224         ],
225     }
226
227     return pd.DataFrame(scenario_rows), risk_frame
228
229 def sobol_analysis(ridge, pchip, design: np.ndarray, sample_power: int = 15) ->
230     tuple[pd.DataFrame, pd.DataFrame]:
231     n = 2**sample_power
232     rng = np.random.default_rng(SEED + 2)
233     a_matrix = rng.random((n, 4))
234     b_matrix = rng.random((n, 4))
235
236     def transform(unit: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray
237     ]:
238         eps = qmc.scale(unit[:, :2], [-TAU, -TAU], [TAU, TAU])
239         return eps, 0.95 + 0.10 * unit[:, 2], 0.95 + 0.10 * unit[:, 3]
240
241     def evaluate(unit: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
242         eps, sm, sq = transform(unit)
243         values = q45.operation_values(design, eps, sm, sq, ridge, pchip,
244             SCENARIOS["S2-A"][0], "A")
245         z = (values - q45.IDEAL) / q45.SPAN
246         dominant = np.argmax(z, axis=1)
247         return q45.score(values), dominant
248
249     y_a, dominant_a = evaluate(a_matrix)
250     y_b, dominant_b = evaluate(b_matrix)
251     variance = float(np.var(np.r_[y_a, y_b], ddof=1))
252     names = ("epsilon_a", "epsilon_b", "s_m", "s_q")
253     rows = []
254     for index, name in enumerate(names):
255         hybrid = a_matrix.copy()
256         hybrid[:, index] = b_matrix[:, index]
257         y_ab, _ = evaluate(hybrid)
258         first_raw = float(np.mean(y_b * (y_ab - y_a)) / variance)
259         first = max(0.0, first_raw)
260         total = float(0.5 * np.mean(np.square(y_a - y_ab)) / variance)

```

```

257     rows.append(
258         {
259             "factor": name,
260             "sobol_first_order": first,
261             "sobol_first_order_raw": first_raw,
262             "sobol_total_order": total,
263             "interaction_gap": total - first,
264             "sample_size": n,
265         }
266     )
267
268     dominant = np.r_[dominant_a, dominant_b]
269     labels = np.array(["thermal", "pressure", "uniformity"])
270     shares = pd.DataFrame(
271         {
272             "dominant_metric": labels,
273             "sample_count": [int(np.sum(dominant == index)) for index in range
274                             (3)],
275         }
276     )
277     shares["sample_share"] = shares.sample_count / len(dominant)
278     return pd.DataFrame(rows), shares
279
280 def main() -> None:
281     pin, base = q3.read_data()
282     ridge, pchip = q3.build_surrogate(pin, base)
283     designs = candidate_designs()
284     distributions = distribution_comparison(designs, ridge, pchip)
285     domain = domain_check()
286     ambiguity, scenario_risks = model_ambiguity_analysis(designs, ridge, pchip)
287     robust_layer = pd.read_csv(STRUCTURE_OPTIMA)
288     robust = robust_layer.iloc[int(robust_layer.CVaR95.argmax())][["a", "b", "N
289                             "]] .to_numpy(float)
289     sobol, dominant = sobol_analysis(ridge, pchip, robust)
290     OUTPUT.mkdir(parents=True, exist_ok=True)
291     outputs = {
292         OUTPUT / "q5_error_distribution_comparison.csv": distributions,
293         OUTPUT / "q5_perturbation_domain_check.csv": domain,
294         OUTPUT / "q5_model_ambiguity_robust.csv": ambiguity,
295         OUTPUT / "q5_cross_scenario_regret.csv": scenario_risks,

```

```
296     OUTPUT / "q5_sobol_indices.csv": sobol,
297     OUTPUT / "q5_dominant_metric_switch.csv": dominant,
298 }
299 for path, frame in outputs.items():
300     frame.to_csv(path, index=False, encoding="utf-8-sig")
301     print(f"wrote {len(frame)} rows to {path}")
302
303
304 if __name__ == "__main__":
305     main()
```